

RAPIDSEGMENT

WHITEBOX RULE MINING ENGINE

A walkthrough of end-to-end rule mining pipeline.

Technical briefing

What we will cover today

01 What RapidSegment is

Purpose and positioning vs black-box models and decision trees

02 End-to-end data flow

From raw table to hierarchical segments and production SQL

03 Binning & variable selection

How features are ranked and discretized (Optimal vs Naive)

04 Combinations & constraints

How 1-/2-/3-way rules are generated, pruned, and filtered

05 Ranking & hierarchy

Champion selection, residual update, final ordered segments

06 Outputs & control levers

SQL filters, scorecard, and the parameters that matter

Transparent segments that maximise lift over baseline

In one sentence

RapidSegment is a combinatorial heuristic engine that discovers high-lift predictive segments and turns them into production-ready SQL rules and scorecards.

It is designed for

- ▶ Credit risk, collections, marketing response, fraud flags
- ▶ Teams that need explainable rules, not black boxes
- ▶ Direct deployment into SQL / policy engines / scorecards

Vs Decision Trees / ML

Goal

High-lift, business-ready segments

Shape

Explicit hierarchical rules + SQL

Reuse

Controlled feature reuse across segments

Output

SQL filters + integer weights

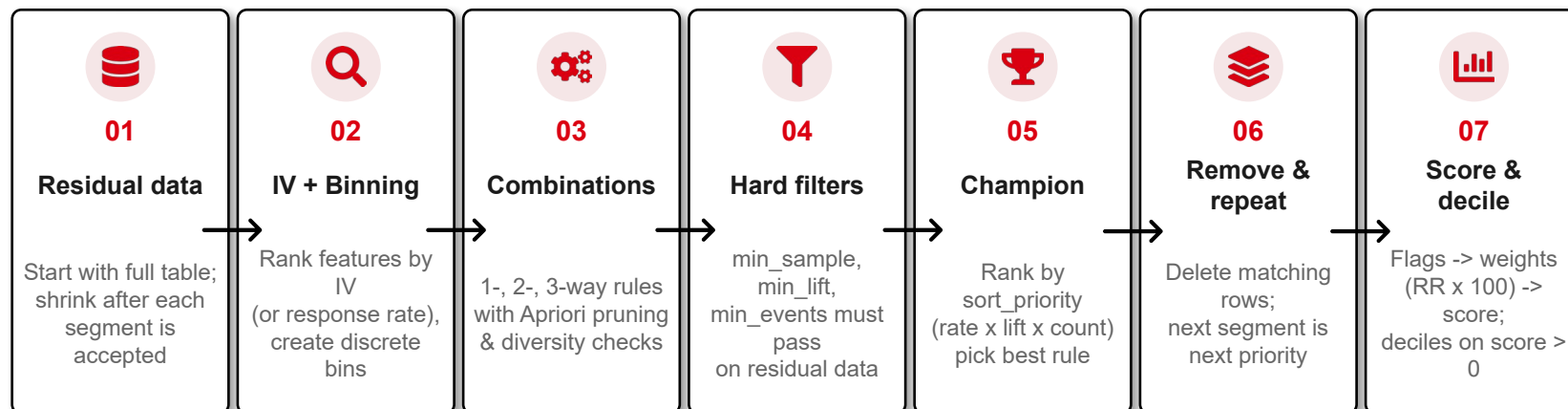
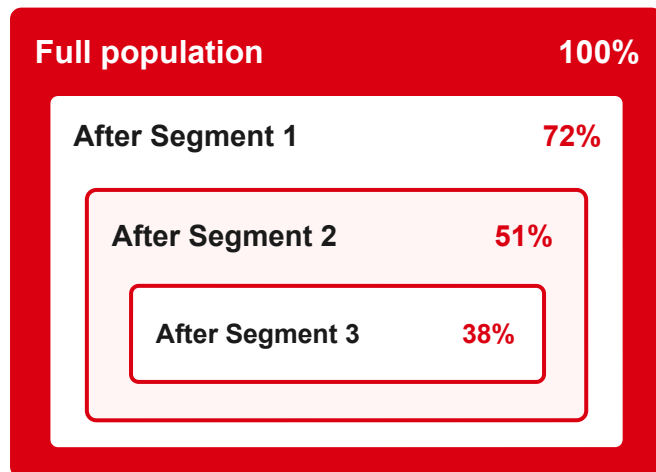
Ops fit

Policy, scorecards, rules engines

From full data to hierarchical segments & scores

RESIDUAL DATA

Starts full · shrinks after each segment



FINAL HIERARCHY

(priority order = discovery order)



KEY IDEA

Find the best rule on what's left -> lock it in -> remove those customers -> repeat.

Then score the full dataset: segment flags x weights (response rate x 100) -> customer score -> deciles on score > 0 only.

Order of discovery becomes priority in the hierarchy (SQL CASE WHEN). Scoring is independent of residual order.

Binning -> Combinations -> Hard constraints -> Ranking -> Residual shrink -> Hierarchy -> Score & decile

Turn raw features into ranked, discrete building blocks

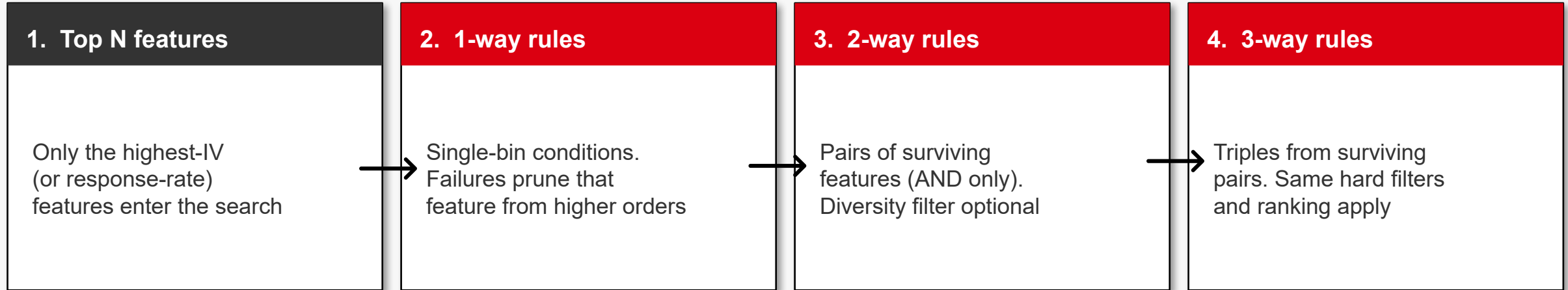
Optimal Binning (default)

Engine	optbinning (CART or Quantile pre-binning)
Metric	Information Value (IV) or Response Rate per feature
Numeric	Numeric bins are auto merged to optimize IV
Categorical	Categories are auto merged to optimize IV
Strength	Target-aware cuts → usually higher lift
When	Accuracy of cuts matters more than speed

Naive Binning (fast path)

Engine	Pure DuckDB SQL (QUANTILE_CONT)
Metric	Information Value (IV) or Response Rate per feature
Numeric	Equal-frequency quantiles (5,10,20...)
Categorical	One bin per distinct value + Missing
Strength	Very fast; supports adjacent-bin expansion
When	Large data or stable, interpretable bins

Apriori-style search: only successful lower-order rules grow



Apriori-style pruning working [example](#)

Design choices that keep search trackable and business-friendly

AND only

OR would break Apriori pruning and explode the search space

Feature groups

Optional diversity: block rules that mix variables from the same business group

max_feature_reuse

Caps how many times one feature can appear across the final segment list

Expansion hops

Optional merge of neighbouring bins to recover higher-event rules

Every candidate must clear absolute floors before ranking



min_sample_size

Minimum row count in the residual that the rule must cover. Protects against tiny, unstable cells.



min_lift

Rule response rate must be at least this multiple of the current residual base rate.



min_events

Minimum number of positive target events inside the rule. Ensures statistical credibility.



Feature reuse / diversity

max_feature_reuse and optional group diversity stop the same variables dominating every segment.

sort_priority decides which surviving rule wins each iteration

Default and common ranking keys (first dimension is primary)

rate_lift_count	Default — prefer highest response rate, then lift, then volume
lift_count_rate	Classic lift-first view of portfolio impact
count_lift_rate	Volume-first when coverage is the business priority
events_lift_rate	Event-count first (good for rare-event targets)
rate_events_lift	High purity + event mass

Grid search (optional)

When param_grid is supplied, the engine evaluates several (min_sample_size, min_lift) pairs per iteration. Each pair produces a candidate; the global champion is still selected by the same sort_priority. This lets the model adapt thresholds as the residual base rate and volume change.

Order of discovery = priority order in the final assignment

1

Accept champion

Rule is stored with SQL filter, count, rate, lift measured on residual

2

Remove matches

Residual updated with NULL-safe logic: WHERE NOT (rule) OR rule IS NULL

3

Next iteration

IV, bins, combinations re-computed on the smaller residual

4

Stop conditions

max_segments reached, or no rule clears hard constraints, or volume too low

5

Final coverage

CASE WHEN hierarchy applied back to original data for true counts

Segments → Scores → Deciles

1



Full dataset
Score every row on the original population (not the residual).

2



Segment flags
Each segment SQL runs independently → binary column (0 / 1) per segment.

3



Segment weights
Weight = segment response rate × 100, rounded. Higher rate → higher weight.

4



Customer score
Score = sum of weights for every segment flag that is 1 on that row.

5



Deciles (score > 0 only)
Split active scores into 10 buckets. Zero-score rows are excluded from thresholds.

WORKED EXAMPLE

Segment weights (from response rate)

Segment	Resp. rate	Weight
Seg 1	18%	18
Seg 2	12%	12
Seg 3	9%	9

One customer example

Seg 1 flag	1	+ 18
Seg 2 flag	0	+ 0
Seg 3 flag	1	+ 9

Total score = 18 + 0 + 9 = 27

Deciles are cut only on customers with score > 0.
Score = 0 stays in the baseline / inactive pool.

Production artefacts from a single `extract_segments` run



SQL filters

Pure ANSI SQL WHERE clauses for every segment. Ready for rules engines, BI tools, or batch scoring.



Hierarchical list

Ordered segments with count, response rate, lift, and meta (thresholds that produced the rule).



Scorecard weights

StrategicSegmentScore converts flags into integer weights from response rate and calibrates deciles.



Full audit trail

`explain_feature_journey`, `explain_no_segments`, feature health reports — complete diagnostics.

Full Flexibility on Segment Construction & Selection

top_n_vars

How many features enter the combinatorial search (default 15)

max_segments

Hard cap on the length of the hierarchy

min_lift / min_sample_size / min_events

Absolute quality floors — the “hard constraints”

param_grid

Sweep alternative floors; champion still chosen by sort_priority

sort_priority

Business preference: rate-first, lift-first, volume-first, events-first

binning_method

optimal_cart / optimal_quantile / naive

max_feature_reuse

Prevent one variable from owning every segment

enable_diversity + feature_groups

Force cross-group combinations only

max_expansion_hops

Allow neighbouring bins to merge (0 = off)

SUMMARY

Five things to remember

- 1 Binning + IV/RR selects a shortlist of predictive features and turns them into discrete bins.
- 2 Combinations are grown bottom-up ($1 \rightarrow 2 \rightarrow 3$ way) with Apriori pruning and optional diversity.
- 3 Hard constraints (sample, lift, events) are absolute gates — ranking only happens among survivors.
- 4 The champion of each residual is kept; matching rows are removed; the next champion is the next priority.
- 5 Output is hierarchical SQL rules + optional scorecard weights — ready for production and audit.

How 1-way, 2-way and 3-way rules are built (dummy data)

Dummy residual · Base rate = 5% | Features ranked by IV: max_dpd · util · risk_band · tenure · product

Hard floors for this example: min_sample = 1,000 · min_lift = 1.5× · min_events = 50

1-WAY

Single-bin conditions

- ✓ **max_dpd ≥ 60**
n=2,400 rate=12% lift=2.4× ✓
- ✓ **util ≥ 0.85**
n=1,800 rate=9% lift=1.8× ✓
- ✓ **risk_band = High**
n=3,100 rate=8% lift=1.6× ✓
- ✗ **tenure < 6m**
n=900 rate=6% lift=1.2× ✗ sample
- ✗ **product = Gold**
n=4,000 rate=5.2% lift=1.04× ✗ lift

Only features that pass
1-way enter 2-way search.

2-WAY

AND of two surviving features

- ✓ **max_dpd ≥ 60 AND util ≥ 0.85**
n=1,200 rate=18% lift=3.6× ✓
- ✓ **max_dpd ≥ 60 AND risk=High**
n=1,500 rate=14% lift=2.8× ✓
- ✓ **util ≥ 0.85 AND risk=High**
n=1,100 rate=11% lift=2.2× ✓
- ✗ ... any pair with tenure
tenure failed 1-way → pruned
- ✗ ... any pair with product
product failed 1-way → pruned

Apriori: failed 1-way features
are never combined further.

3-WAY

AND of three surviving features

- ✗ **max_dpd ≥ 60 ∧ util ≥ 0.85 ∧ risk=High**
n=850 rate=22% lift=4.4× ✗ sample
- ✗ **(no other triple clears floors)**
All other triples fail min_sample
or min_events

In this run the 3-way fails
min_sample → only 1- & 2-way
candidates go to ranking.

[Return to original slide](#)